

Beijing-Dublin International College



AUTUMN TRIMESTER FINAL EXAMINATION - (2025/2026)

School of Computer Science

COMP3038J Programming Exam

Dr. Fergus Toolan
Prof. Neil Hurley
Dr. Seán Russell*

Time Allowed: 180 minutes

Instructions for Candidates:

On the computer choose the image **WIN10-3**
Answer all questions.

BJUT Student ID:_____ **UCD Student ID:**_____

I have read and clearly understand the Examination Rules of both Beijing University of Technology and University College Dublin. I am aware of the Punishment for Violating the Rules of Beijing University of Technology and/or University College Dublin. I hereby promise to abide by the relevant rules and regulations by not giving or receiving any help during the exam. If caught violating the rules, I accept the punishment thereof.

Honesty Pledge:_____ **(Signature)**

Instructions for Invigilators

N/A

Exam Project

To complete this exam you should download the Exam Project zip file from moodle. This file contains a IntelliJ IDEA project with some code for you to use and some input data.

Project Contents

The important files and folders are listed here:

- **customers.csv** - A file containing data about customers
  - Customer id
  - Name
  - Date of Birth
  - Email address
  - Phone number
  - Default currency (CNY, EUR, USD, or GBP)
  - Address - (street, area, city, region, country postcode)
- **items.csv** - A file containing data about the items sold by the store
  - Item id
  - Name
  - Description
  - Rating (All, Adult, or Restricted)
  - Currency (CNY, EUR, USD, or GBP)
  - Price as two integer values units and centiunits (E.g. $12.34 would be 12, 34)
- **stock.csv** - A file containing data about the stock held in the store
  - Item id
  - Amount of that item in stock
- **orders.csv** - A file containing details about the orders that have been made
  - Order ID
  - Customer ID
  - Order Status (New, Processed, AwaitingStock, Collected, Delivered, or Cancelled)
  - Number of items on the order
  - For each of these items there are two entries
    * Item Id
    * Quantity

Rules

- This exam is open book, this means that you are allowed to use the course notes and any textbook during the exam.
- This is also an **individual** exam. All submissions will be compared and any students found to have cheated will be reported to the School of Computer Science plagiarism committee and potentially to the university plagiarism committee.
- The consequences of plagiarism are a best case of **failing** the exam, potential **fines**, **not being allowed** to repeat the module for a year, or a worst case of being **removed from the degree**.
- You should only submit working code. If your code does not compile or work correctly you should remove the part that is not working. You will **fail** if you submit code that **does not compile**.
- The classes you define should have good encapsulation. A portion of your grade in **every** question is based on this.

Submission

In order to complete the exam, you must submit your code to the exam quiz on the class page of csmoodle.ucd.ie. It is your responsibility to ensure that this is done correctly. The quiz close automatically at the end of the exam and will not be reopened for any reason.

If your exam is not submitted correctly and on time you will **fail the exam**.

To do this find the folder containing your project on the computer:

- Right click on the folder and select the option to create ZIP file or compressed file
- Rename the created ZIP file to your **UCD** student number
- Open the Zip file to check your code is inside the file
- Upload the file to moodle

- **DO NOT WAIT UNTIL THE LAST MOMENT TO UPLOAD YOUR SOLUTIONS, NO EXCUSES FOR MISSING THE DEADLINE WILL BE ACCEPTED**
  - **SUBMIT YOUR CODE MULTIPLE TIMES AS YOU COMPLETE THE QUESTIONS**
- **VERIFY THAT YOUR CODE HAS BEEN SUCCESSFULLY UPLOADED BY DOWNLOADING THE FILE AGAIN AND CHECKING THE CONTENTS**

All questions in the exam relate to a inventory and sales management system for an online store. Solutions must be completed in Scala and will be assessed based on their adherence to functional programming principles learned during the course.

Question 1: Inventory and Sales Management System - Modelling

a. Modelling

Based on the description below, define the necessary constructs in Scala to model the elements of the inventory and management system.

The system is being used as a back-end for an international online retailer. The system will manage the data of the customers, inventory, orders, and other related concepts.

Customers

Customers are identified by a unique alphanumeric string and the system records their name, email, date of birth, phone number, address (made up of street, area, city, region, country, and postcode) as well as the default currency they use in their orders. Address should be verified to ensure that street, city, country and postcode are all valid non empty values, when there is a problem with any of these values an appropriate action should be taken. Phone numbers are made up of country code, area code, and number. These should also be verified to ensure that area code is not empty and contains only digits and that number contains only digits and is at least 6 digits in length.

Money

As the store serves markets in different countries, a representation of money in the system is required to reflect this. Any representation of an amount of money should also include the currency that it is represented in. The rates in the table below should be used when conversions between currencies are required. Currently, the store lists items and accepts payments in the following currencies EUR, GBP, USD, and CNY.

Exchange Rates

Currency A	Currency B	A to B	B to A
EUR	USD	1.16206	0.86035
EUR	GBP	0.88181	1.13298
EUR	CNY	8.24901	0.12118
USD	GBP	0.75878	1.31669
USD	CNY	7.09860	0.14085
CNY	GBP	0.10688	9.34663

Inventory

All items available in the shop are identified by a unique alphanumeric string. In addition the following details are remembered for each item; the name of the item, a description of the item, the price of the item (including details of the currency it is in) and an age rating detailing who it may be sold to. The rating `All` means that the item can be purchased by customers of any age, `Adult` means that the item may only be purchased by customers over the age of 18 (on the current date), and `Restricted` means that the item may only be purchased by customers aged 15 or over (on the current date).

The system additionally records the current quantity of each item that is held in the warehouse.

Orders

Each order is identified by a unique alphanumeric string. For each order the system also remembers the customer that made the order, the current status of the order, optionally a postal tracking identifier, as well as the list of items included in the order and the quantity of each being purchased.

When an order is first received, it is given the status `New`. When the order has been packaged and is ready for collection by the carrier the status changes to `Processed` (at this point the stock levels have also been adjusted). If there was not enough stock to complete the order, then the status is `AwaitingStock`. The status `Collected` means the order has been picked up for delivery, and the status `Delivered` means that the order has been confirmed as being received. The status `Cancelled` means that the order was cancelled by the customer, this can not be done if the order is in the `Collected` or `Delivered` status.

Submission

For the majority of the parts of the domain, only the basic definitions of the things to be represented are required. This was called a "skinny domain model" in class.

The exception to this is any part of the code where verification of the parameters of the constructor is required. For these, the code to check the correctness of the parameters and return suitable objects in cases of success and failure should also be included.

(40%)

(Question Total 40%)

Question 2: Inventory and Sales Management System - Functionality

Now that the data has been represented, we need to add some functionality to the system. This code should be implemented based on the "functional objects" approach studied in class.

a. Money Matters

Add code to the system to allow the addition and subtraction of the objects representing money. If these operations are attempted where the two values represent different currencies, then an exception should be thrown. (Something custom or like IllegalArgumentException).

Additionally, add a singleton object that is is responsible for remembering the currency conversion rates and provides functionality to convert a money object to a new currency. The function should throw an IllegalArgumentException if there is no exchange rate available to convert between the two currencies.

(10%)

b. Managing Orders

Code should be added to the system to allow the following updates to the object(s) representing orders in the system:

- The ability to update the status of an order.
- The ability to remove an item from the order.
- The ability to add a quantity of an item to an order. Note: When a quantity of the item already exists in the order the quantity should be updated without adding the same item twice.
- The ability to update the quantity of an item on the order. Note if the quantity of the item is set to 0, then the item should be removed from the order.
- The ability to get the total price of an order in a specified currency.

(10%)

c. System Core

A component is required that will be responsible for remembering all of the data of the system. This should have data structures to remember the customers, items, stock, and orders. In addition to this a number of methods should be added that allow us to add and modify this data.

Define a singleton object to represent this part of the operation of the shop. It should provide methods to achieve each of the following tasks:

- Add a single customer to the system
- Add a single item to the system
- Add a single order to the system
- Update the current stock of a single item
- Return a data structure containing the customer objects
- Return a data structure containing the item objects
- Return a data structure containing the order objects
- Return a data structure containing the items and their current stock

Mutability

The data remembered by this object is expected to be changed frequently during the execution of the program. While the standard practice in functional programming is to only use immutable data, this is a case where it is acceptable to use mutable data structures. **(10%)**

d. Loading Data

Define an object in the system that will be responsible for loading data from a number of csv files. The object should provide a method to achieve each of the following tasks:

- Load the data about customers from a file
- Load the data about items from a file
- Load the data about orders from a file
- Load the data about the current inventory

For each of the above tasks, any errors encountered when processing the data from the file should be output to the screen and only objects read correctly should be returned. **(20%)**

e. Main Program

Define a main method in the file `Main.scala`. In this main method you should complete the following tasks

- Create objects representing the store and the data loader
- Use the methods of the data loading object to retrieve details of customers, items, stock and orders and use the methods of the store object to add them.

(10%)

(Question Total 60%)

Bonus

The following are list of tasks that can be completed in the main function for extra credit. Each task is worth between 2% and 4% and you can earn up to a maximum of 15% in total. Include the text describing the task as a comment immediately before the relevant code.

- Write an expression to create a list containing all of the items that have at least one item in stock and are suitable for a customer under the age of 15
- Write an expression to print the id and price (in CNY) of all of the orders currently in the system
- Write an expression to create a list containing all of the customers that live in either Ireland or China

- Write an expression to print the order id, name of the customer, and country of the customer for all orders that have the status AwaitingStock
- Write an expression to print the name and price and quantity of the 5 items in the store that have the highest amount of stock
- Write an expression to print the total value of all of the items currently in stock in USD
- Write an expression to print the total value in GBP for all orders that have the status AwaitingStock

(Exam Total 100%)